

Prolog is a language with one data structure, one operation and one control strategy: the term, unification, and backtracking search. Everything else follows from those three. This primer walks through how the language works, with examples you can type into the pi interpreter – at its `>` prompt, or in the piview console this document came from. The last section maps the standard onto what pi implements.

01 A program states what is true

A Prolog program is not a list of instructions. It is a set of **facts** and **rules** – a knowledge base – and running a program means asking a **query** against it. The interpreter answers by trying to *prove* the query from what it knows.

```
father(fred,peter).           % a fact: this is simply true
father(fred,mark).
mother(anne,peter).

?- father(fred,peter).        % a query: can this be proven?
yes

?- father(fred,X).            % a query with a variable
X=peter
X=mark
```

A fact ends with a full stop. A query starts with `?-`. When a query contains a variable, the interpreter does not just answer yes or no – it reports every value of the variable that makes the query true. That is the whole model: computation is proof, and answers are the bindings the proof required.

02 Everything is a term

Prolog has a single data structure. Programs, queries, and data are all **terms**, built from four kinds of pieces:

TERM	LOOKS LIKE	WHAT IT IS
<code>fred</code> , <code>ok</code>	lowercase first letter	an atom – a symbolic constant, equal only to itself
<code>42</code> , <code>3.14</code> , <code>-7</code>	digits	a number – integer or float
<code>X</code> , <code>Rest</code> , <code>_</code>	uppercase or <code>_</code> first	a variable – a hole a value can fill; <code>_</code> alone is a fresh anonymous variable every time it appears
<code>father(fred,X)</code>	name + arguments	a compound term – a functor applied to argument terms, written <code>name/arity</code> , here <code>father/2</code>

Lists are compound terms with sugar: `[a,b,c]`, the empty list `[]`, and the pattern `[H|T]` that names a list's first element and the rest of it. Quoted text like `'Move top disk from '` is an atom with spaces in it.

A variable's scope is one clause or one query. Every `X` inside a single rule is the same variable; an `X` in the next rule is a different one. Variables

are not assigned and reassigned – a variable is either *free* or *bound to a term*, and once bound it stays bound for as long as that proof attempt lives.

03 Unification, the one operation

Two terms **unify** when they can be made identical by filling in variables. This single operation is Prolog's parameter passing, pattern matching and construction all at once. The operator `=` asks for it directly:

```
?- X = fred.                % a free variable takes any term
X=fred

?- f(X,b) = f(a,Y).         % both sides can contain holes
X=a
Y=b

?- X = 3 - 1.               % = does not calculate: X is now
X=3-1                      % the term 3-1, not the number 2

?- [H|T] = [1,2,3].         % pattern against list
H=1
T=[2,3]

?- fred = mary.             % two different atoms never unify
no
```

The negation of unifiability is `\=`. It succeeds exactly when its two sides *cannot* be made identical – and it binds nothing: the interpreter attempts the unification, then takes back anything the attempt filled in.

```
?- a \= b.                  % different atoms: not unifiable
yes

?- X \= a.                  % a free variable unifies with
no                          % anything, so this must fail

?- f(X,g(Y)) \= f(a,b), X = c. % the failed attempt bound X to a,
X=c                          % then unbound it again
```

04 How a query runs

A query is a sequence of **goals** joined by `,` (and) or `;` (or). The interpreter works left to right. For each goal it scans the database top to bottom for a clause whose head unifies with the goal. Each match is a **choice point**: the interpreter commits to the first, and if a later goal fails, it **backtracks** – returns to the most recent choice point, undoes the bindings made since, and tries the next match.

```
father(fred,peter).
father(fred,mark).
mother(anne,peter).

?- father(fred,X), mother(anne,X).
X=peter
```

The trace of that query: `father(fred,X)` first matches `father(fred,peter)`, binding `X=peter`. The second goal `mother(anne,peter)` is proven – a solution.

Looking for more solutions, the interpreter backtracks: `X=peter` is undone, `father(fred,X)` re-matches with `X=mark`, but `mother(anne,mark)` fails, and there are no matches left. One solution total.

Failure, in other words, is not an error. It is the ordinary signal that drives the search: try, fail, back up, try the next way.

05 Rules, and rules that use themselves

A rule says: the head is true if the body is. `:-` reads as “if”.

```
parent(tom,bob).
parent(bob,ann).

ancestor(X,Y) :- parent(X,Y).
ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).

?- ancestor(tom,A).
A=bob
A=ann
```

Each rule has two readings, and both are correct. Declaratively: *X is an ancestor of Y if X is a parent of Y, or a parent of someone who is an ancestor of Y.* Procedurally: *to prove ancestor, first try parent; failing that, find a child Z and recurse.* Writing Prolog well is keeping both readings in view at once.

Two clauses with the same head are alternatives – the `;` between them is implicit. The recursion needs no counter and no loop: the list of things left to try *is* the control state.

06 Arithmetic

Because `=` only unifies, arithmetic gets its own operator: `is` evaluates the expression on its right and unifies the result with its left.

```
?- X is 3 - 1.
X=2

?- X is 2 + 3 * 4.           % * and / bind before + and -
X=14

?- X is (2 + 3) * 4.         % brackets override
X=20

?- B is 7 - 2, C is B + 1.    % a bound variable is its value
B=5
C=6

?- 3 is 1 + 2.               % an already-bound left side
yes                           % becomes a check
```

The comparison operators `==` `=\=` `<` `=<` `>` `>=` also evaluate both sides, then compare numbers. They bind nothing.

```

?- 1 + 1 == 2.           % arithmetic equality: evaluates
yes

?- 2 * 3 == 5.           % arithmetic inequality
yes

```

Values computed in a rule's body flow back to its caller like any other binding, so arithmetic composes with recursion:

```

len([],0).
len([_|T],N) :- len(T,M), N is M + 1.

?- len([a,b,c],N).
N=3

```

07 Three ways to ask “equal”

Newcomers trip here, so it deserves its own table. Prolog separates *can these be made the same*, *are these already the same*, and *do these compute the same number*:

ASK	MEANING	X FREE	1 + 1 ... 2
<code>X = T</code>	unify – make identical, binding variables	binds <code>X</code> to <code>T</code>	no – different terms stay different
<code>X == T</code>	identity – already the same term, bind nothing	fails unless <code>T</code> is that same variable	no – <code>1+1</code> is not the term <code>2</code>
<code>X ::= T</code>	evaluate both, compare numbers	fails – nothing to evaluate	yes

Each has its negation: `\=` (not unifiable), `\==` (not identical), `==` (not equal as numbers).

```

?- 1 + 1 == 2.           % identity does not calculate
no

?- X = a, X == a.         % but it sees through bindings
X=a

?- X == Y.                % two distinct free variables
no

?- f(X) == f(X).          % the same variable twice
yes

?- 2 == 2.0.              % an integer is not a float
no

```

08 Lists

A list is walked by splitting it with `[H|T]` and recursing on the tail; it ends at `[]`. The shape of the data dictates the shape of the program – one clause per case:

```
test([]).                % nothing left: done
test([H|T]) :- write(H), nl, test(T).

?- test([a,b,c]).
a
b
c
yes
```

The canonical list predicates of the standard are two-line definitions in the same style. `member/2` is a lesson in nondeterminism – its second clause deliberately keeps looking past the element it found:

```
member(X,[X|_]).         % X is the head, or
member(X,[_|T]) :- member(X,T). % X is somewhere in the tail

?- member(X,[a,b,c]).
X=a
X=b
X=c
```

Because clauses are relations rather than functions, one definition runs in every direction. `append/3` builds a list when its third argument is free – and read backwards, the same two lines *enumerate every way to split one*:

```
append([],L,L).
append([H|T],L,[H|R]) :- append(T,L,R).

?- append([1,2],[3,4],Z).
Z=[1,2,3,4]

?- append(X,Y,[1,2]).
X=[]      Y=[1,2]
X=[1]     Y=[2]
X=[1,2]   Y=[]
```

That is the payoff of the model: `reverse/2`, `last/2` and the rest of the standard library are each a few lines in exactly this shape, and pi ships them in `samples/lists.pl`.

09 The cut

The cut, written `!`, is a goal that always succeeds – and as a side effect throws away the remaining choice points of the clause it stands in. It is how a program says: *this alternative is decided, stop exploring the others*.

```

q(a).
q(b).
q(c).

r(X) :- q(X), !.                % commit to the first q

?- r(X).
X=a

```

Without the cut, `r(X)` would answer three times. With it, the choice points created by `q(X)` are discarded the moment the cut is reached.

A cut that only removes answers the program was going to reject anyway (a *green* cut) is a pure optimisation. A cut whose removal changes the answers (a *red* cut) is program logic hiding in control flow – it has its uses, but every red cut deserves a comment.

10 Negation as failure

`\+ Goal` succeeds exactly when `Goal` cannot be proven. This is not classical negation – it is the *closed world assumption*: what the database cannot prove is taken to be false.

```

q(a).

?- \+ q(z).                % q(z) is unprovable
yes

?- \+ q(a).                % q(a) is provable
no

```

A negated goal binds nothing – whatever the inner proof attempt bound is undone whether it succeeded or failed. That leads to the classic trap: `\+ q(X)` with `X` free does not mean “find an `X` that is not `q`”. It means “is there *no* `q` at all” – the inner goal gets first pick of `X`. Bind your variables before negating them.

11 The wider standard

Full ISO Prolog (ISO/IEC 13211-1) carries more than fits in a small interpreter, and more than fits here. The pieces most worth knowing exist:

- **Collecting solutions** – `findall/3`, `bagof/3` and `setof/3` gather every answer to a goal into a list, turning backtracking into data.
- **A changing database** – `assertz/1` and `retract/1` add and remove clauses from a running program.
- **Term inspection** – `functor/3`, `arg/3` and `=../2` take compound terms apart and build them from lists.
- **Atoms and strings** – `atom_codes/2`, `atom_length/2`, `number_codes/2` convert between atoms, numbers and character lists.
- **Exceptions** – `throw/1` and `catch/3`; standard arithmetic on a non-number throws `type_error` rather than failing.
- **Control** – `->`; (if-then-else), `call/1`, and `true/0`.

12 ISO Prolog in pi

pi implements a compact subset of the standard – every example above not marked otherwise runs in it as printed:

CATEGORY	SUPPORTED
Control	<code>,</code> (and), <code>;</code> (or), <code>:-</code> , <code>!</code> (cut), <code>\+</code> (negation), <code>fail</code>
Unification	<code>=</code> , <code>\=</code>
Identity	<code>==</code> , <code>\==</code>
Arithmetic	<code>is</code> , <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , and <code>:=</code> , <code>\=</code> , <code><</code> , <code>=<</code> , <code>></code> , <code>>=</code>
Lists	<code>[a,b,c]</code> , <code>[]</code> , <code>[H T]</code>
Built-ins	<code>write/1</code> , <code>nl/0</code>

Where pi departs from the letter of the standard:

- A goal is solved to its complete set of solutions in one pass rather than lazily, one answer at a time. The answers are the same; a query with very many solutions holds them all at once.
- No exceptions. Where ISO throws – arithmetic on a non-number, an unbound variable in `is` – pi simply fails.
- The database changes at the prompt, not from programs. There is no `assert/retract`: type a clause to add it, `delete` to remove one, `new` to start over. Agents do the same through piview's MCP tools.
- Recursion is bounded at 1.5M stack entries, reported as “infinite recursion detected” rather than a crash.

The engine itself is described in the project's C4 model, and the interpreter's source is around seven thousand lines of C++ written to be read – the shortest answer to “but how does it *really* work” is in `src/`.